RDBMS 240110102

Unit 4 - Indexing, File Organization, Query Optimization (CO4)

Topics:

File Organization: Heap, Sequential, Hashed , Indexing: Primary, Secondary, Composite, B+ Trees, Cost Estimation Basics, Query Optimization Techniques, Query Execution Plan, Statistics and Histograms

File Organization in DBMS

File organization in DBMS refers to the method of storing data records in a file so they can be accessed efficiently. It determines how data is arranged, stored, and retrieved from physical storage.

The Objective of File Organization

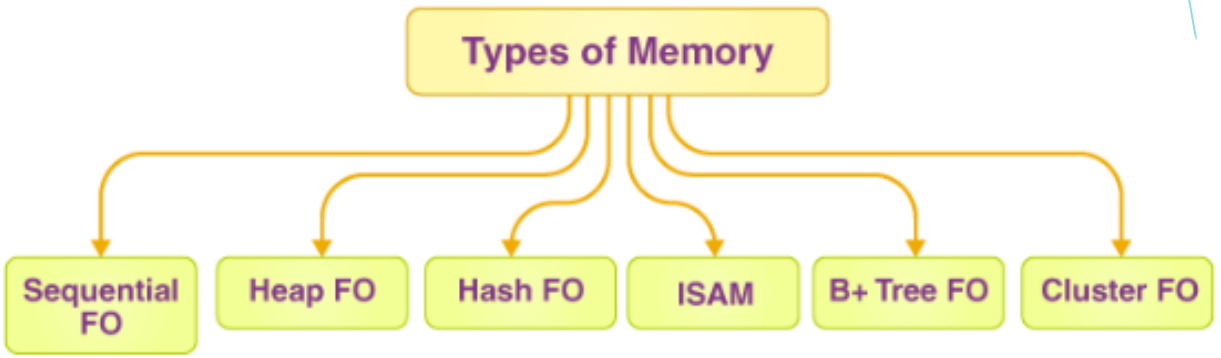
- It helps in the faster selection of records i.e. it makes the process faster.
- Different Operations like inserting, deleting, and updating different records are faster and easier.
- It prevents us from inserting duplicate records via various operations.
- It helps in storing the records or the data very efficiently at a minimal cost.

File Organization in DBMS

Types of File Organizations

Various methods have been introduced to Organize files. These particular methods have advantages and disadvantages on the basis of access or selection. Thus it is all upon the programmer to decide the best-suited file Organization method according to the requirements.

Some types of File Organizations are:



- Sequential File Organization
- Heap File Organization
- Clustered File Organization
- ISAM (Indexed Sequential Access Method)
- Hash File Organization
- B+ Tree File Organization

File Organization in DBMS

Sequential File Organization

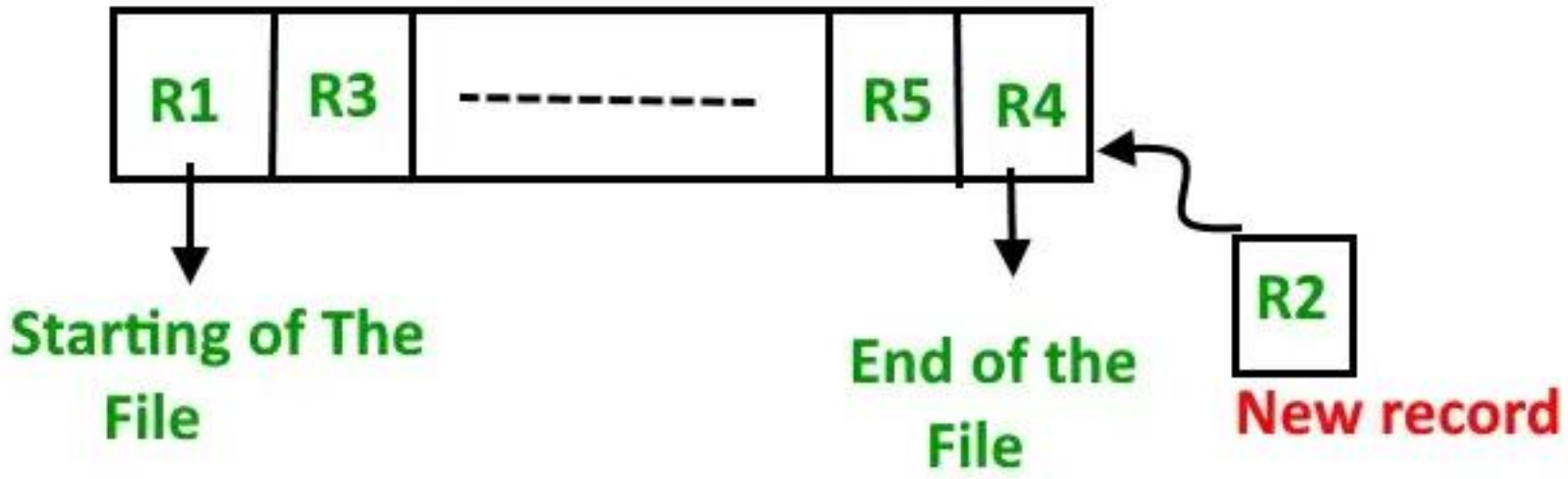
The easiest method for file Organization is the Sequential method. In this method, the file is stored one after another in a sequential manner. There are two ways to implement this method:

1. Pile File Method

This method is quite simple, in which we store the records in a sequence i.e. one after the other in the order in which they are inserted into the tables.

File Organization in DBMS

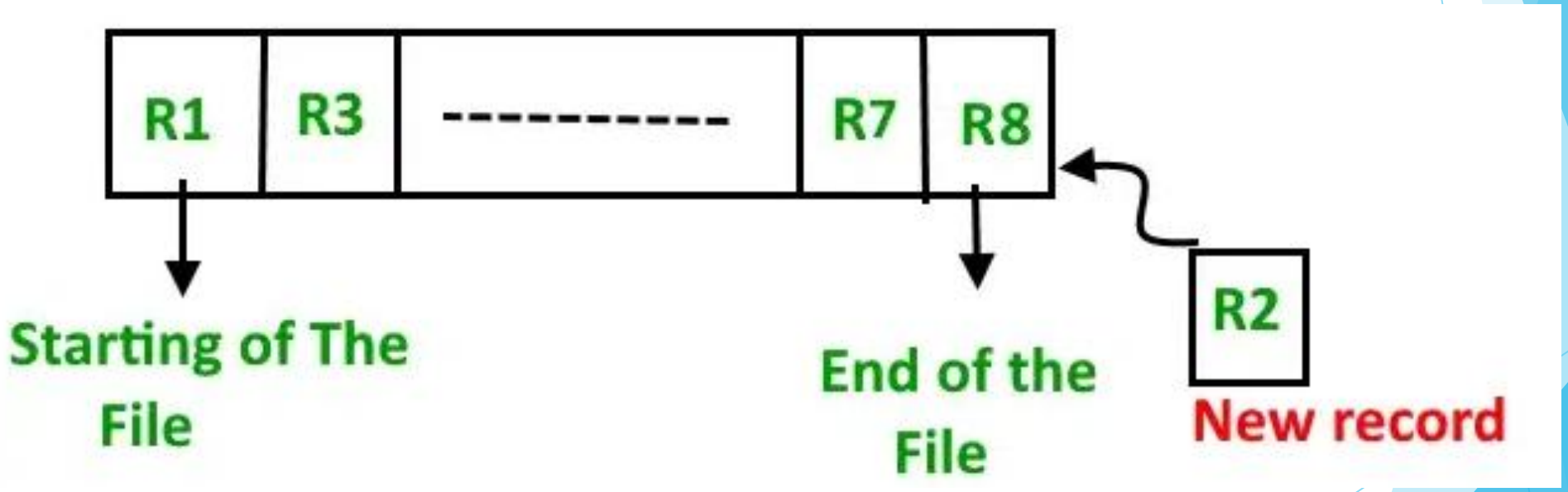
Insertion of the new record: Let the R1, R3, and so on up to R5 and R4 be four records in the sequence. Here, records are nothing but a row in any table. Suppose a new record R2 has to be inserted in the sequence, then it is simply placed at the end of the file.



New Record Insertion

2. Sorted File Method

In this method, As the name itself suggests whenever a new record has to be inserted, it is always inserted in a sorted (ascending or descending) manner. The sorting of records may be based on any primary key or any other key.



File Organization in DBMS

Insertion of the new record: Let us assume that there is a preexisting sorted sequence of four records R1, R3, and so on up to R7 and R8. Suppose a new record R2 has to be inserted in the sequence, then it will be inserted at the end of the file and then it will sort the sequence.

new Record Insertion

File Organization in DBMS

How It Works

1. A **key field** is selected (mostly primary key).
2. Records are **sorted and stored** according to this key.
3. Searching becomes faster because DBMS can use:
    1. Binary Search
    2. Sequential search (in order)
4. Insertions and deletions are harder because the order must be maintained.

Characteristics

- Records are stored in **sorted (ordered)** manner.
- Good for **range queries** (e.g., 100 < Roll No < 200).
- Efficient for **sequential processing.**
- Insertion and deletion require **reorganization** of file.

When to Use Sequential File Organization

It is best when:

- Data is mostly **read**, not updated frequently
- Records need to be processed in sort order
- Range queries are common
- Searching speed is important

Examples:

- Payroll system
- Student result system
- Invoice or billing system

File Organization in DBMS

Advantages of Sequential File Organization

•Fast and efficient method for huge amounts of data.
•Simple design.
•Files can be easily stored in magnetic tapes i.e. cheaper storage mechanism.

Disadvantages of Sequential File Organization

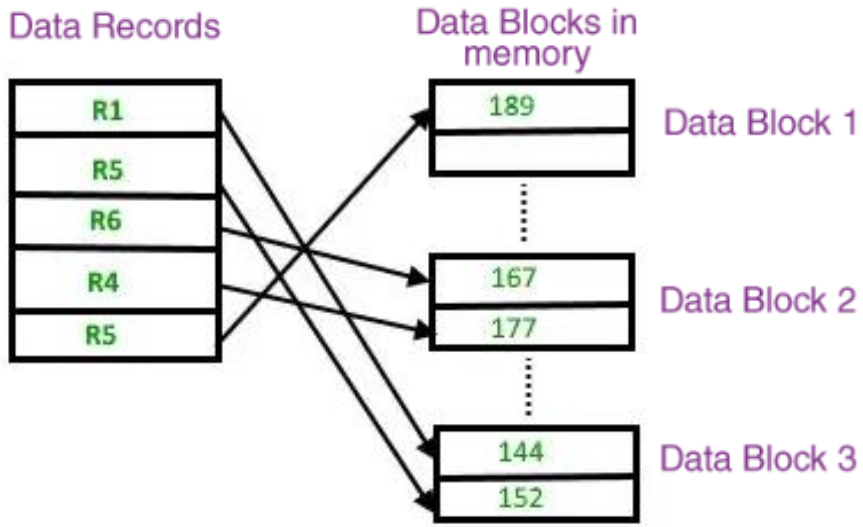
•Time wastage as we cannot jump on a particular record that is required, but we have to move in a sequential manner which takes our time.
•The sorted file method is inefficient as it takes time and space for sorting records.

File Organization in DBMS

Heap File Organization

It is the most fundamental and basic form of file organization. It's based on data chunks. The records are inserted at the end of the file in the heap file organization. The ordering and sorting of records are not required when the entries are added.

The new record is put in a different block when the data block is full. This new data block does not have to be the next data block in the memory; it can store new entries in any data block in the memory. An unordered file is the same as a heap file. Every record in the file has a unique id, and every page in the file is the same size. The DBMS is in charge of storing and managing the new records.

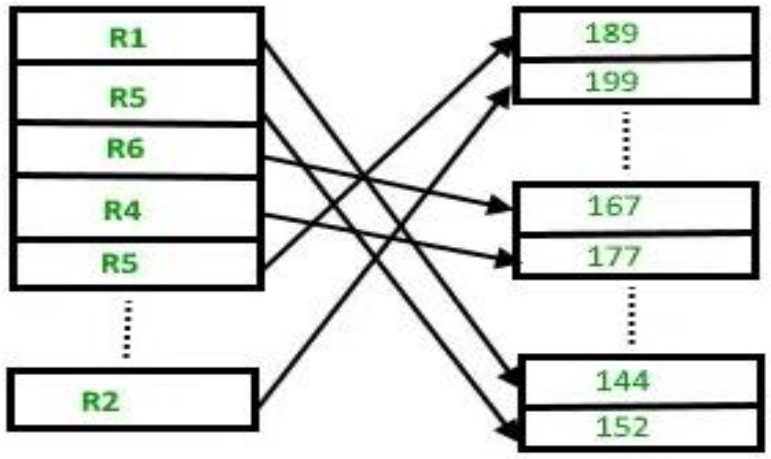


Heap File Organization

File Organization in DBMS

Heap File Organization

Insertion of the new record: Suppose we have four records in the heap R1, R5, R6, R4, and R3, and suppose a new record R2 has to be inserted in the heap then, since the last data block i.e data block 3 is full it will be inserted in any of the data blocks selected by the DBMS, let's say data block 1.



New Record Insertion

If we want to search, delete or update data in the heap file Organization we will traverse the data from the beginning of the file till we get the requested record. Thus if the database is very huge, searching, deleting, or updating the record will take a lot of time.

File Organization in DBMS

How It Works

1. The file is divided into **pages/blocks**.
2. When a new record arrives, DBMS:
    1. Finds the **first available free space** in any block
    2. Stores the record there
3. There is **no sorting or ordering** of records.
4. Searching requires scanning the file from beginning to end.

Characteristics

- Records are **stored randomly**, not sorted.
- Very **fast insertion** (just place it anywhere).
- Searching is **slow**, especially for large files.
- Good when data is inserted frequently.
- Not suitable for range queries.

When to Use Heap File Organization

Use heap organization when:

- Insertion is frequent
- Searching is not frequent
- No ordering is required
- Table size is small
- Temporary or log data

Examples:

- Transaction logs
- Backup tables
- Temporary results
- History tables

File Organization in DBMS

Heap File Organization

Advantages of Heap File Organization

•Fetching and retrieving records is faster than sequential records but only in the case of small databases.
•When there is a huge number of data that needs to be loaded into the database at a time, then this method of file Organization is best suited.

Disadvantages of Heap File Organization

•The problem of unused memory blocks.
•Inefficient for larger databases.

File Organization in DBMS

- Clustered File Organization

Clustered File Organization stores related records from different tables together based on a **common field**, like a **foreign key**. This improves query performance for join operations by placing related data physically close on disk.

A clustered file organization keeps two or more related tables/records in a single file known as a cluster. These files consist of two or more tables within a single data block, and the mapping attributes, defining the relationships between the tables, are stored only once. These contain two or more tables in one data block and the key attributes that are related between the tables are stored only once.

This means it is **cheaper** to search for and retrieve distinct records from different files as they are now integrated and stored in the same cluster

File Organization in DBMS

- Clustered File Organization

How it works

Grouping records: Records from different tables that are frequently joined are stored together in the same data block, forming a cluster.

Cluster key: The cluster is created around a common column, or "cluster key," which is often a primary key in one table and a foreign key in another.

Sorting: The data within the cluster is sorted based on the cluster key, ensuring that records with the same key value are grouped together.

Efficient access: When a query needs data from these joined tables, the database can access the relevant records directly from the cluster without having to search and combine data from separate files.

File Organization in DBMS

- Clustered File Organization

Types of Clustered file organization

1. Indexed Clusters

The clusters are organized, such that records in an indexed cluster are stored in the order of the clustering key. However, for searching and retrieving data, an index is created on the clustering key along with physical sorting as well. When using the range queries and equality searches on the clustering key, this kind of clustering is beneficial.

2. Hash Clusters

In a hash cluster, every record is located in accordance with a hash function on the clustering key. By applying it, one can identify records with the same hash value, and therefore identify their physical location. Another great advantage of hash clustering is that by using their clustering key, it is possible to obtain an individual record very quickly. Still, since records are not sorted in a particular order, the algorithm is slower for range queries.

File Organization in DBMS

- Clustered File Organization

Benefits

Faster queries: Significantly speeds up queries that involve joining the clustered tables, as related data is physically co-located.
Reduced search cost: Eliminates the need for the database to search and access multiple files for a single query.
Direct operations: Allows for direct insertion, update, and deletion of records through join operations.

Example

Imagine you have two tables: **Departments** and **Employees**. Employees are frequently joined with their department information. By using clustered file organization on the department_id (the cluster key), the records for employees in the 'Sales' department could be stored together in a cluster, and all records for the 'HR' department could be stored in a separate cluster. This is more efficient than storing the two tables separately and then performing a join operation every time you need to retrieve employee and department information together.

File Organization in DBMS

- ISAM (Indexed Sequential Access Method)

ISAM (Indexed Sequential Access Method) is a file organization technique that allows records to be accessed both sequentially and randomly. It uses a pre-structured index, which is a sorted list of keys, to map directly to the location of records in a data file, enabling fast retrieval. While ISAM was originally developed for disk storage, it is used in database management systems to provide a balance of sequential and direct access for applications with relatively static data.

File Organization in DBMS

- ISAM (Indexed Sequential Access Method)

How ISAM works

Data File: Records are stored in the data file in a sorted order based on a primary key.

Index File: A separate index file contains entries that are also sorted by the key. Each index entry is a pointer to the physical location (address) of the corresponding record or block in the data file.

Overflow Area: A separate area handles new records that are inserted and don't fit into the primary data area, also organized in a sorted manner.

Searching: To find a record, the system first searches the index to quickly locate the correct data block address, and then it retrieves the record from that location.

File Organization in DBMS

- ISAM (Indexed Sequential Access Method)

Working Mechanism of ISAM

Searching: When searching for a record, the system first looks in the index to find the disk block that contains the record. Then, that specific block is fetched to retrieve the record. This usually takes two disk I/O operations.

Insertion: If a new record needs to be inserted, it is placed in its correct sorted position. If the block is full, the record is placed in the overflow area.

Deletion: Deleting a record involves marking it as deleted. The space is then reclaimed during a subsequent reorganization of the data file.

Updating: Updating a record involves either modifying it if the key is not changed, or deleting and re-inserting it if the key is modified.

File Organization in DBMS

- ISAM (Indexed Sequential Access Method)

Advantages of ISAM

Fast retrieval: ISAM provides fast random access by using the index to directly locate records, which is much quicker than reading through an entire file.
Supports range and partial searches: The sorted nature of the index allows for efficient searching of ranges of data or records that start with a specific string (partial search).

Disadvantages of ISAM

Static structure: The static nature of the index means that the structure of the index tree remains fixed after creation.
Performance degradation: Frequent insertions, deletions, and updates can create "overflow chains," which are scattered records that need to be searched sequentially and can significantly slow down performance.
Inefficient for dynamic data: Because of the overflow chain issue, ISAM is not ideal for databases that undergo constant and rapid changes.

File Organization in DBMS

- **ISAM (Indexed Sequential Access Method)**

Example:

An ISAM example is a simplified library database with a data file of books sorted by ISBN and a separate index file. The index file contains each ISBN and a pointer to the block address in the data file where the book's information is stored. To find a specific book, the system first uses the index to find the correct block address in the data file, then retrieves the data from that location.

File Organization in DBMS

- **ISAM (Indexed Sequential Access Method)**

Example:

Data File: Contains book records, sorted by ISBN

123	Book A
234	Book B
345	Book C
456	Book D

Index File: Maps ISBNs to block addresses in the data file.

ISBN	Block Address
123	1
234	2
345	3
456	4

File Organization in DBMS

- **Hash File Organization**

Hash File Organization is a method of storing and managing records in a database by using a **hash function** to determine the direct memory address or disk block location of each record. This allows for very fast access, insertion, and deletion of individual records, making it a highly efficient technique for applications requiring quick lookups.

File Organization in DBMS

- **Hash File Organization**

Hash Function: A mathematical algorithm that takes a record's key value (e.g., an employee ID) as input and converts it into a specific memory address or a relative bucket number.

Hash Key/Column: The attribute of the record (often the primary key) to which the hash function is applied.

Data Buckets: The memory locations (typically one or more contiguous disk blocks) where the records are physically stored. The hash function maps keys to these buckets.

Collision: A situation where the hash function generates the same address for two different keys. Handling collisions is a critical part of hash file organization

Advantages

✓ Very fast **equality search** (exact match queries)
✓ Direct access to record
✓ Efficient for large number of inserts
✓ Works best for primary key lookups

Disadvantages

✗ Not suitable for **range queries** (e.g., find marks between 70–90)
✗ Bucket overflow handling needed
✗ Hash function must be chosen carefully
✗ Rehashing is costly

Where is Hash File Organization Used?

✓ Bank account search (Account No → record)
✓ Employee ID lookup
✓ Student Roll No lookup
✓ Aadhaar number lookup
✓ URL shortener systems
✓ Hash indexes (like in PostgreSQL, Oracle)

Types of Hashing

Hashing in file organization is generally categorized into two main types:

Static Hashing

In static hashing, the number of data buckets in memory remains constant, and the hash function always produces the same address for a given key.
Collision Resolution Methods: When a collision occurs in static hashing, various techniques are used:

Open Addressing (Closed Hashing): Probing for the next available empty slot in the hash table, using methods like linear probing, quadratic probing, or double hashing.

Separate Chaining (Open Hashing): Each bucket holds a pointer to a linked list that stores all records which hash to that same address

Dynamic Hashing

Dynamic hashing, also known as extendible hashing, allows the data buckets to expand or shrink dynamically as the number of records in the database increases or decreases. This approach is better suited for databases with frequent changes in size and helps maintain good performance without requiring a full rehashing of the entire file

Static vs Dynamic Hashing – Real-Life Summary Table

Feature	Static Hashing (Real Life)	Dynamic Hashing (Real Life)
Bucket size	Fixed hostel blocks	Hostel adds new blocks as students increase
Growth	Overflow adjustments only	Structure grows automatically
Flexibility	Low	High
Handling overload	Add temporary beds	Build a new room/floor
Similar to	Fixed parking space	Parking area expands on demand
Hash behaviour	Same distribution always	Hash structure changes (splits)

Static Hashing → "Fixed drawers in your cupboard"
•If full, clothes spill over or are stuffed in awkwardly
•Structure doesn't grow
Dynamic Hashing → "Cupboard with attachable new drawers"
•If one drawer fills, you add a new drawer
•System adjusts itself

Hash File Organization

How it works

- A hash function is applied to a record's key (e.g., an ID number) to generate a number.
- This number directly points to the disk block where the record is stored.
- When a record is inserted, the hash function determines its block location, and the record is placed there directly.
- To retrieve, update, or delete a record, the same hash function is used to find its address, allowing for direct access to the record without searching.

Hash File Organization

Advantages

Speed: Provides very fast data retrieval because records are accessed directly without a linear search.
Efficiency: Ideal for large databases that require rapid access, such as online banking or e-commerce systems.
Concurrency: Can handle multiple transactions simultaneously, as records are independent and can be accessed at the same time.

Disadvantages

Collisions: Two different keys can sometimes produce the same hash address, leading to a "collision." This requires a secondary method to resolve, such as checking the next available slot, which can slow down operations.

Static hashing: If the database grows or shrinks, a static hash file can become inefficient. It may lead to too many overflows or wasted space if the initial allocation was incorrect. .

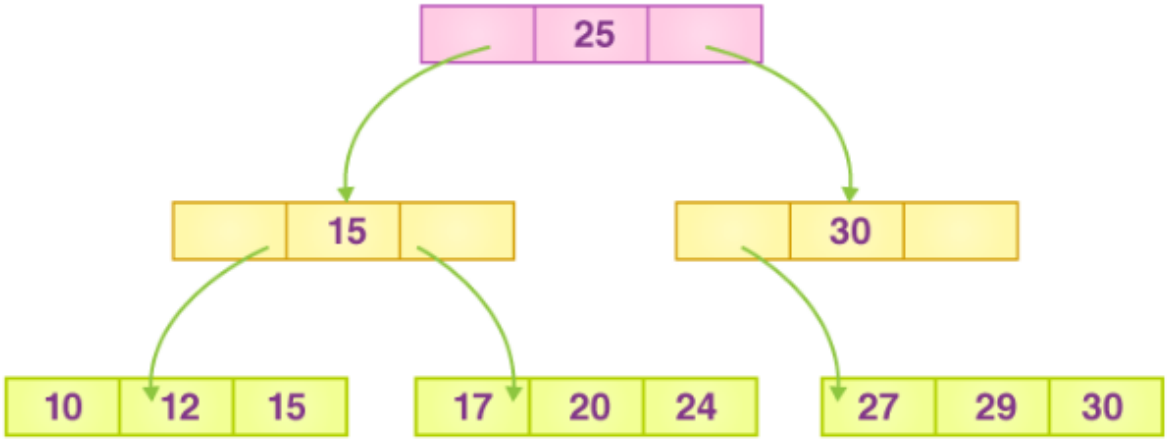
B+ File Organization

The advanced way of an indexed sequential access mechanism is the B+ tree file organization. In File, records are stored in a tree-like structure.

The B+ tree file organization is a very advanced method of an indexed sequential access mechanism. In File, records are stored in a tree-like structure. It employs a similar key-index idea, in which the primary key is utilised to sort the records. The index value is generated for each primary key and mapped to the record.

Unlike a binary search tree (BST), the B+ tree can contain more than two children. All records are stored solely at the leaf node in this method. The leaf nodes are pointed to by intermediate nodes. There are no records in them.

B+ File Organization



- The tree has only one root node, which is number 25.
- There is a node-based intermediary layer. They don't keep the original record. The only thing they have are pointers to the leaf node.
- The prior value of the root Is stored in the nodes to the left of the root node, while the next value is stored in the nodes to the right, i.e. 15 and 30.
- Only one leaf node contains only values, namely 10, 12, 17, 20, 24, 27, and 29.
- Because all of the leaf nodes are balanced, finding any record is much easier.
- **This method allows you to search any record by following a single path and accessing it quickly.**

B+ File Organization

Advantages:

Because all records are stored solely in the leaf nodes and ordered in a sequential linked list, searching becomes very simple using this method.

It's easier and faster to navigate the tree structure.

The size of the B+ tree is unrestricted; therefore, the number of records and the structure of the B+ tree can both expand and shrink.

It is a very balanced tree structure. Here, each insert, update, or deletion has no effect on the tree's performance.

Disadvantages:

The B+ tree file organization method is very inefficient for the static method.

File Organization in DBMS

DBMS looks at the type of operations you (or the application) perform:

Requirement	Best File Organization
Search by primary key	Indexed or Clustered
Range search (BETWEEN, >, <)	B+ Tree (Clustered index)
Insert-heavy table	Heap or Sequential
Very fast equality search	Hash
Mostly read-only table	Sequential

✓ **Heap Example**
Teacher inserts attendance log entries daily:
Order does not matter → heap structure fits.

✓ **Sequential Example**
Bank maintains a file of customers **sorted by account number**.

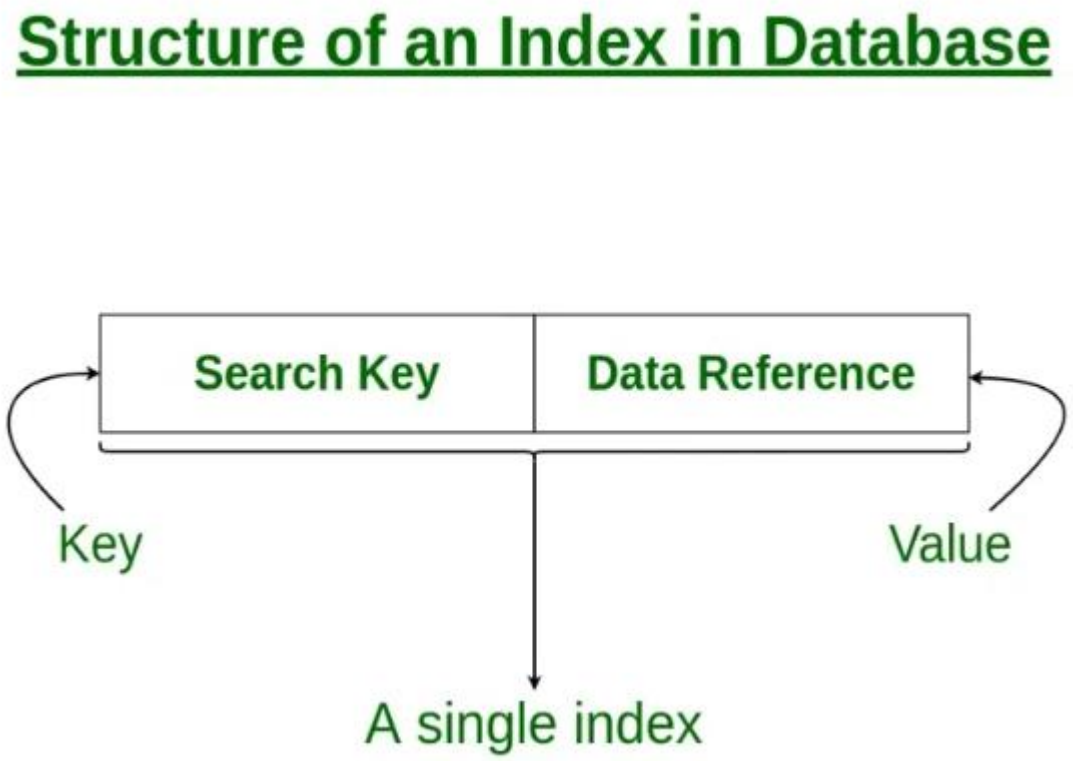
✓ **Clustered Example**
Hospital file clustered by "Department":
Cardiology patients stored together physically.

✓ **Hash Example**
Retrieve patient record by Aadhaar number (exact match).

✓ **Indexed Example**
Library book search by Title/Author using indexes.

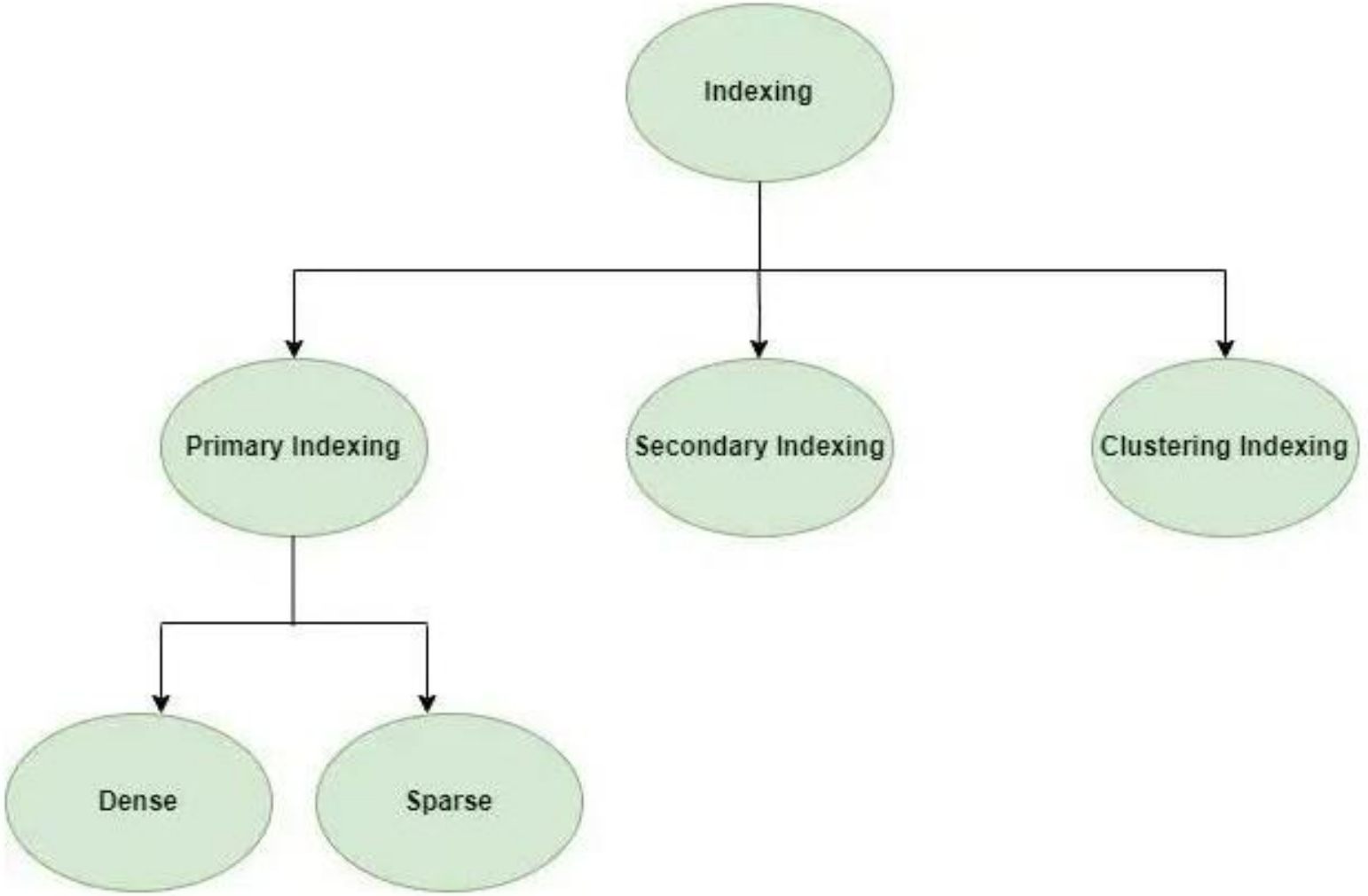
Indexing in Databases

Indexing in DBMS is used to speed up data retrieval by minimizing disk scans. Instead of searching through all rows, the DBMS uses index structures to quickly locate data using key values.
When an index is created, it stores sorted key values and pointers to actual data rows. This reduces the number of disk accesses, improving performance especially on large datasets.



Indexing in Databases

Structure of Index in Database



Structure of Index in Database

Indexing in Databases

Types of Indexing Methods

1. Primary Indexing

This is a type of Clustered Indexing where in the data is sorted according to the search key and the **primary key** of the database table is used to create the index. It is a default format of indexing where it induces sequential file organization. As primary keys are **unique** and are stored in a sorted manner, the performance of the **searching operation** is quite efficient.

Key Features: The data is stored in sequential order, making searches faster and more efficient.
Indexing

Indexing in Databases

Types of Indexing Methods

There are different types of indexing techniques, each optimized for specific use cases.

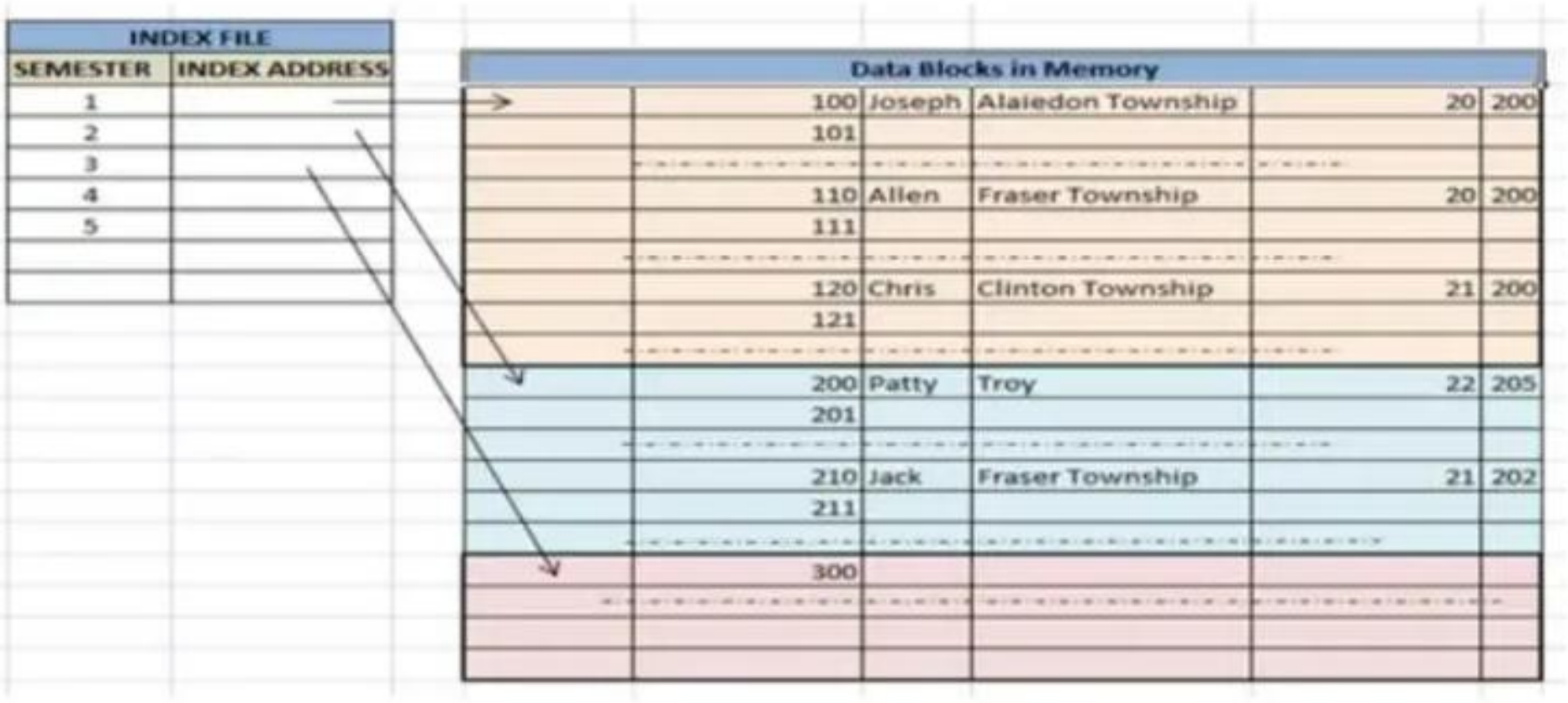
2. Clustered Indexing

Clustered Indexing stores related records together in the same file, reducing search time and improving performance, especially for join operations. Data is stored in sorted order based on a key (often a **non-primary key**) to group similar records, like students by semester. If the indexed column isn't unique, multiple columns can be combined to form a unique key. This makes data retrieval faster by keeping related records close and allowing quicker access through the index.

Indexing in Databases

Types of Indexing Methods

Clustered Indexing



Clustered Indexing

Indexing in Databases

Types of Indexing Methods

3. Non-clustered or Secondary Indexing

A non-clustered index just tells us where the data lies, i.e. it gives us a list of virtual pointers or references to the location where the data is actually stored. Data is not physically stored in the order of the index. Instead, data is present in leaf nodes.

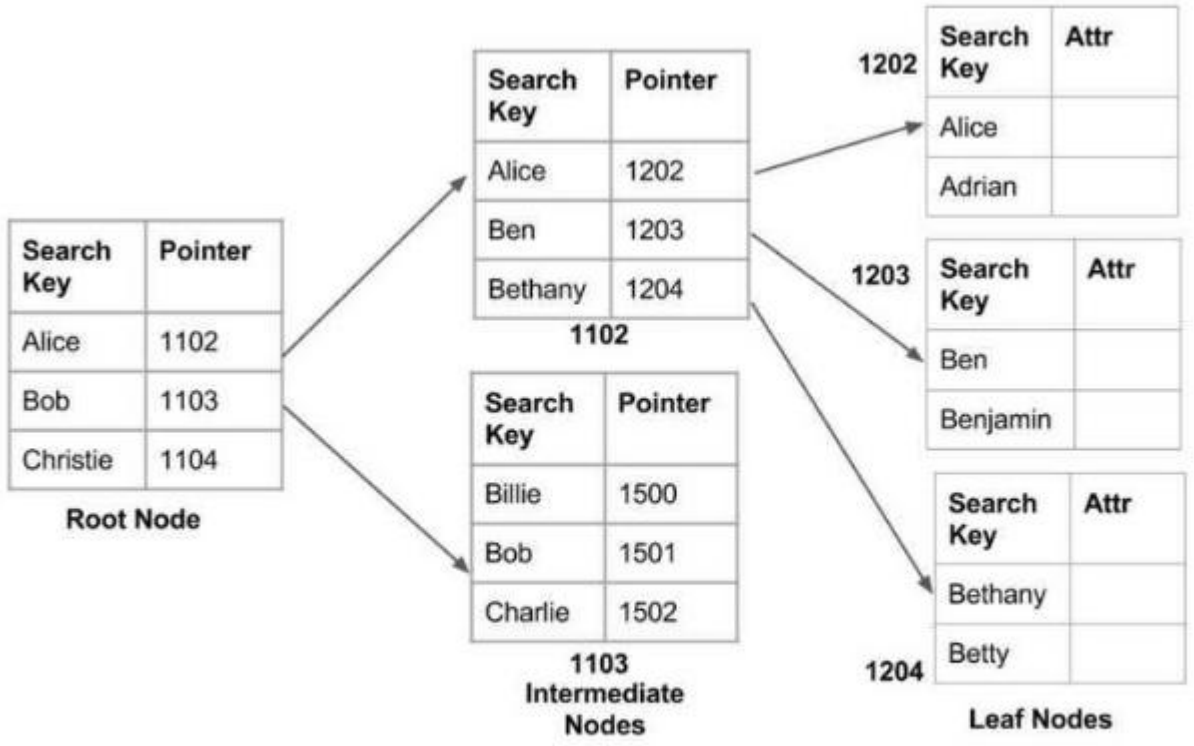
Example: The contents page of a book. Each entry gives us the page number or location of the information stored. The actual data here(information on each page of the book) is not organized but we have an ordered reference(contents page) to where the data points actually lie. We can have only dense ordering in the non-clustered index as sparse ordering is not possible because data is not physically organized accordingly.

It requires more time as compared to the clustered index because some amount of extra work is done in order to extract the data by further following the pointer. In the case of a clustered index, data is directly present in front of the index.

Indexing in Databases

Types of Indexing Methods

3. Non-clustered or Secondary Indexing



Non clustered index

Indexing in Databases

Advantages of Indexing

Faster Queries: Indexes allow quick search of rows matching specific values, speeding up data retrieval.

Efficient Access: Reduces disk I/O by keeping frequently accessed data in memory.

Improved Sorting: Speeds up sorting by indexing the relevant columns.

Consistent Performance: Maintains query speed even as data grows.

Data Integrity: Ensures uniqueness in columns indexed as unique, preventing duplicate entries.

Indexing in Databases

Disadvantages of Indexing

While indexing offers many advantages, it also comes with certain trade-offs:

Increased Storage Space: Indexes require additional storage. Depending on the size of the data, this can significantly increase the overall storage requirements.

Increased Maintenance Overhead: Indexes must be updated whenever data is inserted, deleted, or modified, which can slow down these operations.

Slower Insert/Update Operations: Since indexes must be maintained and updated, inserting or updating data takes longer than in a non-indexed database.

Complexity in Choosing the Right Index: Determining the appropriate indexing strategy for a particular dataset can be challenging and requires an understanding of query patterns and access behaviours.

.

Cost Estimation Basics in DBMS

Cost estimation means calculating **how expensive** a query is for the database to execute.
Database systems try to **choose the fastest and cheapest way** to run a query.
The cost usually depends on:

1 Disk I/O (Most Important)

•Reading data from a disk is slow.
•Most cost formulas are based on:
- Number of blocks read
- Number of blocks written

•Disk I/O dominates total cost because it is MUCH slower than CPU operations.
Example:
If a table has **1000 blocks**, and a query performs a **full table scan**,
→ **Cost = 1000 block reads**

2 CPU Cost

•Execution of operations like:
- Comparison
- Join condition checking
- Sorting

•Usually small compared to disk cost.

3 Network Cost (in distributed systems)

•Data transfer between nodes.
•Not relevant for single-machine DBMS.

DBMS sometimes ignores CPU cost and considers **only disk cost** for approximation.

What the DBMS Needs Before Estimating Cost

To calculate cost, DBMS uses:

4 Cardinality

Number of tuples (rows) in a table.

Example:

Table STUDENT has **10,000 rows** → cardinality = 10,000

5 **Selectivity**

Fraction of rows that satisfy the condition.

Example:

Query: SELECT * FROM STUDENT WHERE gender = 'F';

If 40% are female → selectivity = 0.4

Selectivity formula:

```
Selectivity = number of matching rows / total rows
```

6 Size of a tuple and size of a block

•Determines how many tuples fit into a block.

•Used for disk I/O calculations.

Example:

Block size = 1 KB

Tuple size = 100 bytes

→ Each block contains 10 tuples

Cost Estimation Examples

A. Cost of Full Table Scan

If table has B blocks, full scan cost = B block reads

Example:

500-block table → cost = 500 I/Os

B. Cost of Using an Index

Index reduces search cost.

Example:

Index height = 3 (**Index Height in DBMS** refers to the **number of levels** in an index structure (usually a **B-Tree or B+ Tree**) from the **root node to the leaf node**. Lower height = faster lookup, Higher height = slower lookup, Each level adds **one disk access**.)

→ Cost = 3 I/Os to reach leaf node

•few more to fetch data blocks.

C. Cost of Join Operations

Each join type has a different cost:

1. Nested Loop Join

Cost = M + (M × N)

(M, N are number of blocks of two tables)

2. Sort-Merge Join (Sort–Merge Join is a join algorithm used when two relations (tables) must be combined using a join condition**)**

Cost = Sorting cost + merging cost

3. Hash Join

Cost = building hash + probing hash

Why Cost Estimation is Needed?

DBMS uses cost estimation to **choose the best execution plan** for a query.

Example:

```
SELECT * FROM EMP WHERE age = 25;
```

Two possible methods:

1. Full table scan → 5000 I/Os
2. Index scan → 5 I/Os

DBMS picks method 2 because its **cost is lowest**.

Query Optimization Techniques (DBMS)

Query Optimization is the process of selecting the *most efficient* way to execute a SQL query. The DBMS optimizer evaluates many possible query execution strategies and picks the one with **minimum cost** (I/O, CPU, time).

There are **two main types** of optimization techniques:

1. Heuristic (Rule-Based) Optimization

Heuristic = "Rule of thumb."

These are general rules used to **rearrange** or **simplify** the query before executing.

⭐ **Common Heuristic Rules:**

1 Push Selection Down

Apply selection (WHERE conditions) **as early as possible**.

Example:

Instead of scanning whole table then filtering, filter first.

```
SELECT * FROM EMP WHERE dept = 'CSE';
```

Filter dept='CSE' early → reduces number of rows → reduces cost.

2 Push Projection Down
Select only required columns early.
Example:
If query needs **only name**, don't keep all columns until last.
👉 Reduces tuple size → fewer disk blocks needed.

3 Use Join Ordering
Join smaller tables first (if possible).
(A JOIN B JOIN C)
Better to join two small tables before joining a big one.

4 Replace Cartesian Product + Filter with JOIN
Instead of:

```
FROM A, B WHERE A.id = B.id
```

Use:

```
A JOIN B ON A.id = B.id
```

Faster and less memory usage.

5 Use Indexes
If a column is indexed, optimizer prefers index scan instead of full table scan.

6 Remove Unnecessary DISTINCT, ORDER BY
If not required, avoid them—they are expensive operations.

2. Cost-Based Optimization

This is the **main technique used by modern DBMS** (MySQL, PostgreSQL, Oracle).
The optimizer:
1 **Generates multiple possible execution plans**
2 **Estimates the cost of each plan** using:
•**of disk I/Os**
•CPU time
•Selectivity
•Cardinality
•Index costs
•Join method cost

3 **Chooses the plan with lowest estimated cost**

Components Used in Cost-Based Optimization

1 Selectivity
Fraction of rows matching condition.
2 Cardinality
Number of rows in each relation.
3 Join Algorithms
Optimizer chooses:
•Nested Loop Join
•Hash Join
•Sort-Merge Join
4 Access Paths
Choose between:
•Table scan
•Index scan
•Index-only scan

3. Transformation-Based Optimization

Query is converted into equivalent forms using **relational algebra rules**:

- Selection pushdown
- Join commutativity
- Join associativity ($(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$ -- **re-associate** (regroup) the join operations.)
- Projection pushdown

Purpose → reduce size of intermediate results.

Notes:

Join Commutativity is a transformation rule used in **query optimization** that allows the optimizer to **reorder join operations** without changing the final result of the query.

A **join operation is commutative** if:

$R \bowtie S = S \bowtie R$

This means **the order of relations in a join can be swapped**, and the output will still be the same (same tuples, though the physical order may differ).

Since join commutativity allows **R join S** to be rewritten as **S join R**, the optimizer can:

✓ Choose the smaller table as the outer table
✓ Pick a more efficient join method
✓ Reduce intermediate result sizes
✓ Generate multiple valid join trees for cost comparison

4. Physical Optimization Techniques

DBMS also selects **physical operations:**

1 Join Methods

•Nested loop join

•Block nested loop join

•Hash join

•Merge join

2 Sorting Methods

•External merge sort

•In-memory sort

3 Index Choices

•B-tree index

•Bitmap index

•Hash index

Simple Example of Optimization

```
SELECT name
FROM EMP
WHERE age = 25 AND salary > 50000;
```

DBMS will optimize like:

1. Push selections down
2. Use index on age or salary if available
3. Fetch only name (projection pushdown)
4. Choose best join or scan method

Result → Fastest execution plan

What is a Query Execution Plan?

A **Query Execution Plan (QEP)** is a **step-by-step blueprint** that shows **how the database will execute a SQL query.**
It is generated by the **Query Optimizer** and shows:

•Which operations will be used
•In what order
•Which indexes or join methods will be chosen
•Estimated cost of each step
Think of it as the **roadmap** for executing a query efficiently.

Why is a Query Execution Plan needed?

Because a single SQL query can be executed in **many ways.**
Example:
•Use index or scan the whole table?
•Join using nested loop, hash join, or sort-merge join?
•Which table to join first?
•How to push selections/projections?
The plan helps the DBMS choose the **fastest** and **lowest-cost** method.

How is a Query Execution Plan created?

1.Parsing

The SQL query is checked for syntax errors.

2.Logical Plan Generation

Query is converted into relational algebra (e.g., σ, π, ⋈).

3.Query Optimization

Many possible plans are generated using:

1. join commutativity
2. join associativity
3. selection pushdown
4. projection pushdown

4.Cost Estimation

Each possible plan gets an estimated cost based on:

1. number of disk I/Os
2. CPU usage
3. size of intermediate results
4. available indexes

5.Best Plan Selected

The plan with the **lowest estimated cost** becomes the **Execution Plan**.

What does an Execution Plan contain?
A QEP includes:

Component	Meaning
Access Path	How the table is accessed (index scan or full scan)
Join Order	Which table is joined first
Join Method	Nested loop, hash join, sort-merge join
Predicate pushdown	Applying filters early
Projection pushdown	Selecting only required columns early
Estimated cost	Approx disk + CPU cost
Estimated cardinality	Number of rows expected

Example of a Query Execution Plan

```
SELECT E.name, D.departmentName
FROM Employee E
JOIN Department D ON E.deptId = D.id
WHERE salary > 50000;
```

A possible plan:

1. **Index Scan on Employee(salary)** – apply salary filter
2. **Hash Join** Employee with Department
3. **Projection** – pick only required columns

Types of Query Execution Plans

1. Logical Query Plan
It represents the query using **relational algebra operations** like selection, projection, and join.
It focuses on *what* operations will be performed, not *how* they will be executed.
2. Physical Query Plan
This plan specifies the **actual algorithms and access methods** (index scan, hash join, sort) used to execute the query.
It shows *how* the DBMS will perform each operation physically.
3. Estimated Execution Plan
Generated before running the query, it provides **estimated cost, cardinality, and chosen operations**.
It helps predict query performance without executing it.
4. Actual Execution Plan
Produced after running the query, showing the **real cost, row counts, and actual operations used**.
It is used for performance debugging and tuning.

How to view QEP in real DBMS?

Oracle

```
EXPLAIN PLAN FOR
SELECT * FROM table_name;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```